

# **MANUAL TÉCNICO**

## **Sistema Experto para Diagnóstico Básico de Fallas en Redes**

Juan Sebastián García Redondo

David Alexander Acosta Pineda

Ashlee Valentina Tapia Medina

Versión 0.1  
Bogotá D.C., Colombia – Julio 2026

## 1. Introducción y alcance

Este manual técnico está dirigido a desarrolladores, ingenieros de conocimiento y personal técnico responsable de instalar, operar, mantener y extender el Sistema Experto para Diagnóstico Básico de Fallas en Redes. Complementa al Informe Técnico del proyecto (que documenta el fundamento teórico y los resultados) con instrucciones operativas concretas.

El sistema, en su versión actual, es un prototipo académico compuesto por cinco artefactos independientes que se documentan en este manual: una base de conocimiento en Prolog, una ontología en OWL, un grafo de conocimiento, un dataset de entrenamiento y dos notebooks de Google Colab con los modelos de Machine Learning.

## 2. Arquitectura general del sistema

La Figura 1 presenta la arquitectura general propuesta para la versión completa del sistema (incluyendo el componente conversacional basado en LLM, aún no implementado en este prototipo). Los componentes ya implementados y documentados en este manual son: la base de conocimiento en Prolog, el motor de inferencia de Prolog, la ontología/grafó de conocimiento y los modelos de Machine Learning.

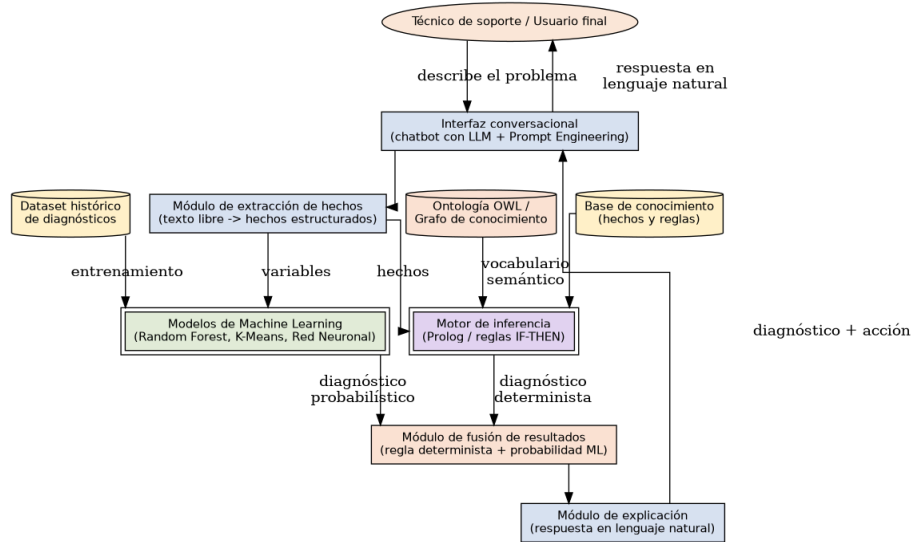


Figura 1. Arquitectura general integrada del sistema.

La Figura 2 detalla específicamente la arquitectura interna del sistema experto (módulo central del sistema).

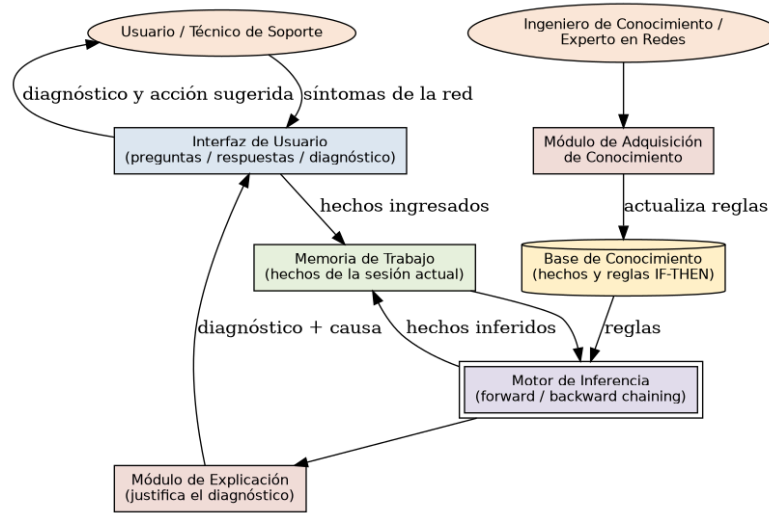


Figura 2. Arquitectura interna del sistema experto (Prolog).

## 2.1 Estructura de archivos del proyecto

```

proyecto-sistema-experto-redes/
├── diagnostico_red.pl                # Base de conocimiento (Prolog)
├── ontologia_diagnostico_red.owl     # Ontología OWL (Protégé)
├── dataset_diagnostico_red.csv       # Dataset de entrenamiento (600 casos)
├── modelo_supervisado_diagnostico_red.ipynb # Árbol de Decisión y Random Forest
├── modelo_no_supervisado_red_neuronal.ipynb # K-Means y Red Neuronal (MLP)
├── images/                          # Diagramas de arquitectura y grafo
├── Informe_Tecnico_Sistema_Experto_Redes.docx # Informe técnico completo
├── Manual_Usuario_Sistema_Experto_Redes.docx # Este documento (usuario)
└── Manual_Tecnico_Sistema_Experto_Redes.docx # Este documento (técnico)

```

### 3. Requisitos de instalación

| Componente                  | Requisito                                  | Versión recomendada                        |
|-----------------------------|--------------------------------------------|--------------------------------------------|
| Motor de inferencia         | SWI-Prolog                                 | 9.0 o superior                             |
| Ontología                   | Protégé Desktop                            | 5.6 o superior                             |
| Modelos de Machine Learning | Python 3 + cuenta de Google (Google Colab) | Python 3.10+                               |
| Librerías Python            | pandas, numpy, scikit-learn, matplotlib    | Ver requirements en 3.1                    |
| Sistema operativo           | Windows, macOS o Linux                     | Cualquiera con soporte para los anteriores |

#### 3.1 Dependencias de Python

```
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
matplotlib>=3.7
```

*Nota: Google Colab ya incluye todas estas librerías preinstaladas; no se requiere instalación manual si se ejecutan los notebooks en ese entorno.*

## 4. Instalación y puesta en marcha

### 4.1 Base de conocimiento en Prolog

1. Instalar SWI-Prolog desde <https://www.swi-prolog.org/download/stable>.
2. Abrir una terminal y ubicarse en la carpeta del proyecto.
3. Cargar la base de conocimiento con el comando: `swipl diagnostico_red.pl`
4. Verificar la carga correcta ejecutando una consulta de prueba, por ejemplo: `?- diagnostico(pc_marketing, Falla).`
5. El intérprete debe responder con el diagnóstico correspondiente (Falla = `error_configuracion_dns`).

### 4.2 Ontología en Protégé

1. Instalar Protégé Desktop desde <https://protege.stanford.edu>.
2. Abrir Protégé y seleccionar Open desde el menú File.
3. Seleccionar el archivo `ontologia_diagnostico_red.owl`.
4. Verificar en la pestaña Classes que se hayan cargado las 5 clases (Dispositivo, Usuario, Falla, Regla\_Diagnostico, Accion\_Correctiva).
5. Verificar en la pestaña Object Properties las 5 relaciones (`presentaFalla`, `requiereAccion`, `reportadaPor`, `conectadoA`, `diagnosticadaPor`).
6. Verificar en la pestaña Individuals by class los 10 individuos definidos.
7. (Opcional) Ejecutar un razonador como Hermit o Pellet desde el menú Reasoner para validar la consistencia lógica de la ontología.

### 4.3 Notebooks de Google Colab

1. Abrir Google Colab en <https://colab.research.google.com>.
2. Seleccionar Upload notebook y cargar `modelo_supervisado_diagnostico_red.ipynb` (o el notebook no supervisado).
3. Subir también el archivo `dataset_diagnostico_red.csv` usando el panel de archivos del lado izquierdo, o mediante el código `from google.colab import files; files.upload()` incluido comentado en la primera celda.
4. Ejecutar las celdas en orden con Entorno de ejecución > Ejecutar todas.
5. Los resultados (métricas, gráficas) se generarán automáticamente en las últimas celdas del notebook.

## 5. Descripción técnica de los componentes

### 5.1 Base de conocimiento (Prolog)

El archivo `diagnostico_red.pl` contiene 10 hechos base y 5 reglas de inferencia, estructuradas como cláusulas de Horn. El predicado principal expuesto a los consumidores del sistema es:

```
diagnostico(+Dispositivo, -Falla)
```

Donde `Dispositivo` es un átomo que identifica el equipo a diagnosticar (por ejemplo, `pc_finanzas`) y `Falla` es la variable de salida que el motor de inferencia unifica con el diagnóstico correspondiente, si existe una regla aplicable. El motor de inferencia de SWI-Prolog resuelve la consulta mediante resolución SLD (backward chaining), evaluando las reglas en el orden en que fueron declaradas en el archivo.

### 5.2 Sistema experto de 15 reglas

El diseño lógico completo de las 15 reglas de diagnóstico (condición IF, diagnóstico THEN y acción sugerida) se encuentra documentado en el Capítulo 6, Tabla 2, del Informe Técnico del proyecto. Para incorporar estas 15 reglas al motor Prolog en un entorno de producción, cada regla debe traducirse a una cláusula equivalente a las ya presentes en `diagnostico_red.pl`, siguiendo el patrón:

```
diagnostico(D, NombreDelDiagnostico) :-
    condicion_1(D),
    condicion_2(D).
```

### 5.3 Ontología OWL

La ontología fue serializada en formato RDF/XML (estándar OWL) y puede editarse directamente en Protégé o mediante cualquier librería compatible con OWL, como Owlready2 en Python, para su integración programática con el resto del sistema.

### 5.4 Dataset

| Campo             | Tipo       | Valores posibles                            |
|-------------------|------------|---------------------------------------------|
| tipo_conexion     | Categorico | ethernet, wifi                              |
| led_enlace_router | Categorico | apagado, rojo, parpadeante, verde           |
| ip_asignada       | Categorico | ninguna, autoconfigurada_apipa, valida_dhcp |
| ping_router       | Categorico | exitoso, fallido                            |
| ping_dns_externo  | Categorico | exitoso, fallido                            |
| resolucion_dns    | Categorico | funciona, falla                             |

|                        |                       |                                                                              |
|------------------------|-----------------------|------------------------------------------------------------------------------|
| acceso_web             | Categorico            | nulo, parcial, total                                                         |
| autenticacion_usuario  | Categorico            | correcta, rechazada                                                          |
| diagnostico (objetivo) | Categorico (9 clases) | falla_enlace_fisico, fallo_dns, aislamiento_firewall, sin_falla, entre otras |

## 5.5 Modelos de Machine Learning

| Modelo             | Librería / clase                        | Hiperparámetros principales                              |
|--------------------|-----------------------------------------|----------------------------------------------------------|
| Árbol de Decisión  | sklearn.tree.DecisionTreeClassifier     | max_depth=5, random_state=42                             |
| Random Forest      | sklearn.ensemble.RandomForestClassifier | n_estimators=200, max_depth=8, random_state=42           |
| K-Means            | sklearn.cluster.KMeans                  | n_clusters=9, n_init=10, random_state=42                 |
| Red Neuronal (MLP) | sklearn.neural_network.MLPClassifier    | hidden_layer_sizes=(6,4), activation=relu, max_iter=2000 |



## 6. Cómo extender el sistema

### 6.1 Agregar una nueva regla de diagnóstico

1. Identificar los nuevos hechos (síntomas) necesarios y declararlos como predicados en `diagnostico_red.pl`.
2. Escribir la nueva regla siguiendo el patrón descrito en la sección 5.2, asignándole un nombre de diagnóstico único.
3. Añadir la acción correctiva correspondiente en la documentación del sistema experto (Tabla 2 del Informe Técnico).
4. Volver a cargar el archivo en SWI-Prolog y probar la nueva regla con una consulta de ejemplo.

### 6.2 Agregar una nueva clase o relación a la ontología

1. Abrir `ontologia_diagnostico_red.owl` en Protégé.
2. Crear la nueva clase, individuo o relación desde la interfaz gráfica de Protégé.
3. Ejecutar el razonador (Reasoner > Start reasoner) para validar que no se introdujeron inconsistencias lógicas.
4. Exportar y sobrescribir el archivo `.owl` (File > Save).

### 6.3 Reentrenar los modelos de Machine Learning con nuevos datos

1. Añadir los nuevos casos (filas) al archivo `dataset_diagnostico_red.csv`, manteniendo el mismo formato de columnas.
2. Volver a ejecutar el notebook correspondiente (`modelo_supervisado_diagnostico_red.ipynb` o `modelo_no_supervisado_red_neuronal.ipynb`) de principio a fin.
3. Comparar las nuevas métricas (Accuracy, F1-Score) contra los valores documentados en el Capítulo 8 del Informe Técnico antes de reemplazar el modelo en producción.

## 7. Mantenimiento y monitoreo

- Revisar periódicamente los casos que el sistema experto no logra diagnosticar (ninguna regla aplicable) para identificar patrones que ameriten una nueva regla.
- Monitorear el desempeño (Accuracy / F1-Score) del modelo de Machine Learning en producción; una caída sostenida puede indicar data drift y la necesidad de reentrenamiento.
- Versionar los archivos `diagnostico_red.pl`, `ontologia_diagnostico_red.owl` y `dataset_diagnostico_red.csv` en un repositorio de control de versiones (Git) para mantener trazabilidad de los cambios.
- Documentar cada nueva regla o clase añadida, indicando la fecha, el autor y la justificación del cambio.

## 8. Solución de problemas técnicos

| Problema                                      | Causa probable                                             | Solución                                                                                            |
|-----------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| SWI-Prolog no reconoce el archivo .pl         | Ruta de archivo incorrecta o codificación no UTF-8         | Verificar la ruta con pwd/cd y guardar el archivo en codificación UTF-8.                            |
| Una consulta no retorna resultados (false)    | Los hechos requeridos por la regla no fueron declarados    | Revisar que todos los hechos necesarios estén presentes antes de la consulta.                       |
| Protégé marca la ontología como inconsistente | Conflicto entre restricciones de clases o relaciones       | Ejecutar el razonador y revisar la pestaña Explanation para identificar el axioma conflictivo.      |
| Error de importación de librería en Colab     | Librería no instalada en el entorno                        | Ejecutar !pip install nombre_libreria --quiet en una celda antes de la importación.                 |
| Baja exactitud al reentrenar el modelo        | Dataset insuficiente o desbalanceado tras la actualización | Verificar el balance de clases con value_counts() y considerar técnicas de balanceo (oversampling). |